Chapter 8

TOWARDS THE GEOMETRIC WORKBENCH

Before we go deeply into the development ot the Geometric WorkBench, it is useful to take a bird's-eye-view look into its basic ideas and its architecture, and give an outline of how the following chapters relate to its overall structure.

8.1 DESIGN GOALS OF GWB

Solid modeling systems are certainly fairly nontrivial programs, and ideally their construction should be based on very well understood methods and a very clear overall conception of the software architecture of the modeler. Because we shall attempt to describe the construction of a solid modeler in the limited space available in a few chapters of a book, these requirements are even more important for GWB.

What characterizes a "good" software design, then, and what design and development methods should be followed to guarantee the quality of the result? These problems form the nutshell of the field of *software engineering*, and a reader not familiar with it can certainly benefit from getting acquaintained with software engineering literature (e.g., [113] or the "classic" [20]).

While many software design and development methodologies have been described, and some even applied succesfully in practical work, a focal point of the various alternative approaches can be labeled with the term *modu-*

larity. A complicated problem can only be solved if it can be broken into simpler subproblems that can be solved separately, and later combined to yield the total solution. To allow for division of labor in software design and development, the subproblems should be relatively independent and have simple, "narrow" interfaces to each other. If the components of a software system have these properties, they are said to exhibit "good modularity."

Another key expression of software engineering methodology is *abstraction.* The inner workings of a significant software system can be exceedingly complicated, and a human being cannot perceive the logic of its operation if he/she attempts to understand all its details at once, much less design or implement it. When breaking a problem down into modular subproblems, it is therefore vital to define the intended operation of each component in abstract terms, and consider the internal details of its implementation separately from the abstract external specification.

Quite properly, the development of various *abstraction mechanisms* for the specification and implementation of software modules has been one of the major subareas of software engineering. It is beyond the scope of this book to review these mechanisms in detail here. Suffice it to say that a major goal of the various abstraction mechanisms is the *information hiding*: a good module should appear opaque to the external viewer as far its internal operation is concerned, and certainly not exhibit surprising side effects.

When applied to a collection of modules, one particular approach to achieve information hiding is the division of the software system into *layers*, each layer consisting of modules with a clear abstract meaning such that the layer forms a functionally complete "abstract machine." If this approach is strictly followed, layer i will use the facilities made available by the layer $i-1$, but has no access to any of the layers $i-2$, $i-3$, and so on. In turn, layer $i+1$ will be based on the external properties of layer i.

8.2 ARCHITECTURE OF GWB

In the design of GWB, we shall make an attempt to follow the general principles outlined in the previous section. In particular, we shall try to distuingish between separable layers of modules, each forming a functionally complete "modeling machine" suitable for the implementation of the next layer.

8.2.1 Layers of GWB

The overall software architecture of GWB forms the onion-like structure depicted in Figure 8.1. In this architecture, the innermost layer will consist

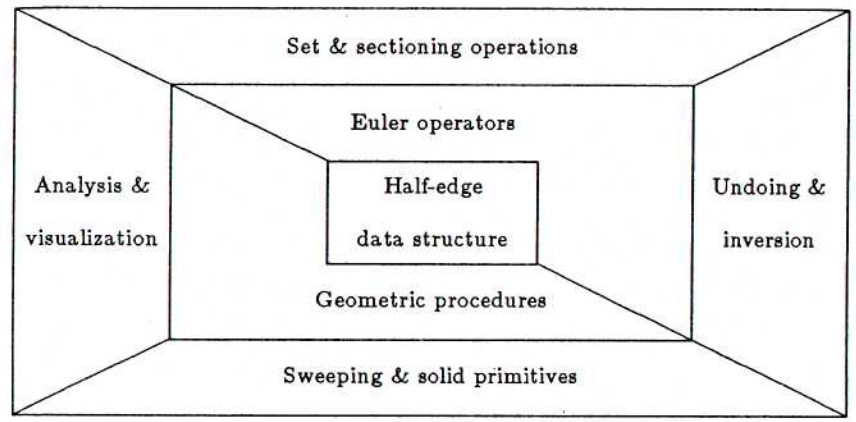

Figure 8.1 Architecture of GWB.

of the internal data representation of GWB, the *half-edge data structure*, and a set of procedures for its low-level manipulation. The half-edge data structure is a variation of the full winged-edge data structure outlined in Chapter 6. The half-edge data structure is described in detail in Chapter 10.

The next layer will consist of the so-called *Euler operators* to be described in Chapters 9 and 11, and a set of basic geometric procedures to be described in Chapter 12. As to be discussed in more detail later, Euler operators form the basic modeling procedures of GWB that will be used to implement all higher-level operations.

The next layer consists of various "medium-level" modeling tools that make the description of meaningful models more straightforward. These procedures implement a collection of local model manipulations, such as sweeping and "swinging" (rotational sweeping), and allow the construction of various solid primitives. Without a better term, we shall use the term "model description tools" of these procedures. They are the subject of Chapter 13. The model analysis tools introduced in Chapter 12 also belong to this layer.

The following layer is formed by various "model manipulation tools" such as splitting and sectioning (Chapter 14), and Boolean set operations (Chapter 15).

Chapter 16 will consider a useful extension to the Euler operator layer of Chapter 11 that allows the undoing of arbitrary sequences of Euler op-

erators. This extension makes it possible to construct certain useful model manipulation tools, such as procedures for storing models into external storage (the *inversion algorithm*), and retrieving them back.

Using the functionality developed in the earlier chapters, Chapter 17 will describe the final layer of GWB, namely its *user interface* through which the modeling tools of the various levels finally are made available to a user.

Most of the Part Two will deal with the construction of the basic polyhedral modeler with little attention on storing information on curved surfaces, or on providing interfaces towards analytical modelers. The final Chapter 18 will describe extensions to the basic modeler that make it a proper faceting modeler. With these extensions, the functionality of GWB is sufficient for interactive processing in a graphics workstation, and for exchanging exact information with a distinct analytical modeler.

8.2.2 An Analogy: Computer Languages

A useful analogy to the architecture of GWB may be found from the relationships of various layers of computer languages.

In this analogy, the half-edge data structure and the low-level operations on it have the role of the processing hardware. Indeed, the facilities available in a particular computer—its registers, memories, and data transfer paths, and the machine language operations that manipulate them—determine the scope of operations that can be performed Similarly, all modeling operations will finally be mapped onto operations performed on the data representation.

Of course, it would be very inconvenient to describe nontrivial computations directly in terms of a machine language, and a need for higher-level languages arises. On the next level, the assembly language can be used to describe arbitrary computations in a much more convenient notation, which can be mapped into the machine language with the help of an assembler.

In our analogy, Euler operators take the role of the assembly language. Indeed, they describe operations of the underlying "machinery" in a much more convenient and mnemonic notation than a program written to manipulate the half-edge data structure directly.[1] Nevertheless, they are still of a relatively low level, and should be hidden beneath other, more powerful operations.

The analogy works also on higher levels. Instead for assembly language programming, we prefer to work with high-level languages that give us more natural tools for expressing computations than those available on the

[1] Going one step further, Euler operators are a kind of "machine-independent assembler" in that they can be implemented in similar fashion on the top of many different data structures. Well, let's not try to stretch the analogy too far.

assembly language level. Likewise, we shall provide higher level tools for geometric modeling on the top of the Euler operator level. In particular, the user will never use Euler operators directly, but only the higher-level operations built on top of them.

Just as in the case of high-level languages, many alternatives as for these higher levels exist. While we shall concentrate on certain often used high-level modeling tools, it should be borne in mind that these are not the only possibilities, just as the same computer can be used to run both LISP and COBOL, and anything in between and beyond.

PROBLEMS

8.1. Discuss the benefits of good software engineering practices as for the reuse and maintenance of software.

8.2. The onion-layer structure is just one approach to the construction and management of large software systems. Discuss the potentials of some other approaches you are familiar with (such as object-oriented programming) for a task such as the design and implementation of a solid modeler.